

Systemy Zarządzania Treścią

Projekt zaliczeniowy

Autorzy:

Piotr Eliks 152028

Adam Czyżak 151814

1. Charakterystyka ogólna projektu.....	2
2. Wymagania.....	2
2.1. Aktorzy:.....	2
2.2. Strona dla klientów.....	2
2.2.1. Wymagania funkcjonalne.....	2
2.2.2. Wymagania niefunkcjonalne.....	2
2.3. Panel administracyjny.....	3
2.3.1. Wymagania funkcjonalne.....	3
2.3.2. Wymagania niefunkcjonalne.....	3
3. Architektura systemu.....	4
4. Schemat baz danych.....	5
4.1. Modelu związków encji.....	5
4.2. Model relacyjny.....	5
5. Diagramy UML.....	6
5.1. Diagram przypadków użycia.....	6
5.2. Diagram przebiegów.....	6
5.3. Diagram stanów.....	6
5.4. Diagram klas.....	6
6. Projekty interfejsu graficznego.....	6
7. Najważniejsze metody i fragmenty kodu aplikacji.....	6
8. Testy.....	6
9. Podsumowanie.....	6
9.1. Podział prac / harmonogram.....	6
9.2. Cele zrealizowane, cele niezrealizowane, napotkane problemy.....	6
9.3. Perspektywa rozwoju.....	6

1. Charakterystyka ogólna projektu

Celem projektu jest opracowanie prostego systemu zarządzania treścią (CMS) wraz z powiązaną stroną internetową dla barbershopu. System pozwala administratorowi na wygodne dodawanie, edycję i usuwanie treści w panelu administracyjnym, bez konieczności modyfikowania kodu strony.

Projekt składa się z dwóch głównych elementów:

- **Panel administracyjny** – umożliwia zarządzanie wszystkimi treściami, takimi jak opis salonu, dane kontaktowe, lista pracowników, portfolio usług czy wpisy na blogu.
- **Strona publiczna** – prezentuje zawartość w atrakcyjnej formie, ułatwiając odwiedzającym szybki dostęp do informacji o salonie, pracownikach i oferowanych usługach.

Dzięki wykorzystaniu bazy danych, wszystkie informacje są przechowywane w sposób dynamiczny, co pozwala na ich bieżące aktualizowanie i utrzymanie spójności. System jest elastyczny, a administrator może w prosty sposób wprowadzać zmiany w treściach bez ingerencji w strukturę strony.

2. Wymagania

2.1. Aktorzy:

- Administrator - posiada pełne uprawnienia do realizacji wszystkich funkcjonalności.
- Redaktor - posiada uprawnienia nadane przez administratora do np. tworzenia, edytowania i usuwania treści z wybranych stron.

2.2. Strona dla klientów

2.2.1. Wymagania funkcjonalne

Aktor: Gość odwiedzający stronę - [anonimowy użytkownik przeglądający treści](#):

- Przeglądanie podstawowych informacji o barbershopie, w tym lokalizacji, godzin otwarcia i danych kontaktowych
- Zapoznanie się z pracownikami – ich specjalizacjami i krótkimi opisami
- Przeglądanie galerii portfolio z przykładowymi fryzurami i stylizacjami wykonanymi przez barberów
- Czytanie artykułów dotyczących barberingu, pielęgnacji włosów i trendów w stylizacji

2.2.2. Wymagania niefunkcjonalne

- [Szybkie ładowanie strony i jej elementów, FCP < 3s](#)
- Responsywność – poprawne wyświetlanie na różnych urządzeniach
- Przyjazny i intuicyjny interfejs
- Stabilne działanie bez błędów
- Estetyczna i czytelna prezentacja treści
- [Lazy-loading zdjęć](#)

- Strona stworzona według standardu WCAG 2.1

2.3. Panel administracyjny

2.3.1. Wymagania funkcjonalne

Aktor: Administrator (niezalogowany):

- Logowanie do panelu administracyjnego
- Resetowanie hasła

Aktor: Administrator (zalogowany):

- Tworzenie konta redaktora
- Nadawanie uprawnień dla konta
- Dodawanie, edytowanie i usuwanie treści takich jak opis salonu, pracownicy, portfolio, artykuły blogowe i kontakt
- Zarządzanie multimediami – dodawanie i usuwanie zdjęć klientów oraz plików niezbędnych do strony
- Zarządzanie metadanymi stron
- Zarządzanie strukturą menu
- Zarządzanie wszystkimi stronami

Aktor: Redaktor (niezalogowany):

- Logowanie do panelu administracyjnego
- Resetowanie hasła

Aktor: Redaktor:

- Logowanie do panelu administracyjnego
- Dodawanie, edytowanie i usuwanie treści takich jak opis salonu, pracownicy, portfolio, artykuły blogowe i kontakt zgodnie z nadanymi uprawnieniami
- Zarządzanie multimediami – dodawanie i usuwanie zdjęć klientów oraz plików niezbędnych do strony zgodnie z nadanymi uprawnieniami
- Zarządzanie metadanymi stron zgodnie z nadanymi uprawnieniami
- Zarządzanie wybranymi stronami

2.3.2. Wymagania niefunkcjonalne

- Szyfrowanie haseł, hasła z minimalną długością 8 znaków z jednym znakiem specjalnym, jedną małą i jedną wielką literą
- Weryfikacja dostępu do poszczególnych funkcjonalności
- Intuicyjna obsługa, nie wymagająca wiedzy programistycznej
- Niezawodne działanie przy wprowadzaniu zmian w treściach, operacje dodawania, usuwania i edycji treści nie powinny przekraczać czasu 4 sekund
- Przejrzysta struktura panelu, umożliwiająca łatwą orientację w dostępnych funkcjach

3. Architektura systemu

- Frontend - React - biblioteka języka programowania JavaScript, która wykorzystywana jest do tworzenia interfejsów graficznych aplikacji internetowych.
- Backend - [Node.js](#) - wieloplatformowe środowisko uruchomieniowe o otwartym kodzie do tworzenia aplikacji typu server-side napisanych w języku JavaScript.
- Baza danych - PostgreSQL - relacyjna baza danych do przechowywania danych,
- Konteneryzacja - Docker - oprogramowanie służące do konteneryzacji, działające jako platforma dla programistów i administratorów do tworzenia, wdrażania i uruchamiania aplikacji rozproszonych.
- System kontroli wersji - Git - rozproszony system kontroli wersji, umożliwiające wspólną pracę nad jednym kodem.
- Repozytorium - GitHub - serwis przeznaczony do przechowywania kodu źródłowego.
- Środowisko programistyczne - Visual Studio Code - edytor kodu źródłowego.
- Szablon strony publicznej - <https://themewagon.com/themes/free-bootstrap-4-html5-hair-salon-website-template-hairsal/>
- Szablon panelu administracyjnego - <https://tailadmin.com/>

5. Diagramy UML

5.1. Diagram przypadków użycia

5.2. Diagram przebiegów

5.3. Diagram stanów

5.4. Diagram klas

6. Projekty interfejsu graficznego

7. Najważniejsze metody i fragmenty kodu aplikacji

8. Testy

9. Podsumowanie

9.1. Podział prac / harmonogram

9.2. Cele zrealizowane, cele niezrealizowane, napotkane problemy

9.3. Perspektywa rozwoju